DSA Week 1

Sequences — Arrays & Strings

A Comprehensive Beginner's Guide with Python Solutions

---

12 Problems | 6 Easy | 6 Medium

Complete with detailed explanations, code, examples, and complexity analysis

Table of Contents

Core Problems

#	Problem	Difficulty	Key Pattern
1	Two Sum	Easy	Hash Map
2	Contains Duplicate	Easy	Set
3	Best Time to Buy and Sell Stock	Easy	Running Minimum
4	Valid Anagram	Easy	Frequency Counter
5	Valid Parentheses	Easy	Stack
6	Maximum Subarray	Easy	Kadane's Algorithm
7	Product of Array Except Self	Medium	Prefix/Suffix
8	3Sum	Medium	Sort + Two Pointers
9	Merge Intervals	Medium	Sort + Merge
10	Group Anagrams	Medium	Hash Map Grouping

Optional Problems

#	Problem	Difficulty	Key Pattern
11	Maximum Product Subarray	Medium	Dual Tracking DP
12	Search in Rotated Sorted Array	Medium	Modified Binary Search

Key Patterns You Will Learn

1. **Hash Map / Dictionary** — Store and lookup values in O(1). Used in Two Sum, Group Anagrams, and many more.
2. **Set** — Track unique elements efficiently. Perfect for duplicate detection.
3. **Two Pointers** — Use two indices to scan from both ends. Essential for sorted array problems.
4. **Sliding Window / Running Minimum** — Maintain a running state as you scan. Used in stock prices.
5. **Stack** — LIFO structure for matching, nesting, and reversal problems.
6. **Kadane's Algorithm** — Dynamic programming for maximum subarray problems.
7. **Prefix / Suffix** — Precompute cumulative results from both directions.
8. **Sorting + Greedy** — Sort first, then apply greedy logic. Used in intervals and k-Sum.
9. **Binary Search** — O(log n) search in sorted (or rotated sorted) arrays.
10. **Frequency Counting** — Count occurrences to compare, group, or validate.

1. Two Sum [Easy]

Problem Statement

Given an array of integers **nums** and an integer **target**, return the **indices** of the two numbers such that they add up to the target. You may assume that each input would have **exactly one solution**, and you may not use the same element twice. You can return the answer in any order.

Approach & Explanation

- A brute-force approach checks every pair of numbers — O(n^2) time.
- The optimal approach uses a **hash map (dictionary)** to store numbers we have already seen.
- For each number, we calculate its **complement** (target - current number).
- If the complement is already in our dictionary, we found our answer!
- If not, we store the current number and its index in the dictionary.
- This gives us a single-pass O(n) solution.

Complexity Analysis

Time Complexity: *O(n)* | ***Space Complexity:*** *O(n)*

Python Solution

```
def twoSum(nums, target):
    # Dictionary to store {value: index}
    seen = {}

    for i, num in enumerate(nums):
        complement = target - num

        # Check if complement exists in our map
        if complement in seen:
            return [seen[complement], i]

        # Store current number with its index
        seen[num] = i

    return []  # No solution found

# --- Driver Code ---
nums = [2, 7, 11, 15]
target = 9
print(f"Input:  nums = {nums}, target = {target}")
print(f"Output: {twoSum(nums, target)}")

nums2 = [3, 2, 4]
target2 = 6
print(f"\nInput:  nums = {nums2}, target = {target2}")
print(f"Output: {twoSum(nums2, target2)}")
```

Input / Output Examples

```
Input:  nums = [2, 7, 11, 15], target = 9
Output: [0, 1]
Explanation: nums[0] + nums[1] = 2 + 7 = 9

Input:  nums = [3, 2, 4], target = 6
Output: [1, 2]
Explanation: nums[1] + nums[2] = 2 + 4 = 6
```

Beginner Tip: *Hash maps let you trade space for speed. Instead of searching the entire array for a match, you look it up instantly in a dictionary.*

2. Contains Duplicate [Easy]

Problem Statement

Given an integer array **nums**, return **True** if any value appears **at least twice** in the array, and return **False** if every element is distinct.

Approach & Explanation

- The simplest approach: use a **set** to track numbers we have seen.
- As we iterate through the array, check if the current number is already in the set.
- If yes, we found a duplicate — return True.
- If no, add the number to the set and continue.
- Alternative shortcut: compare len(nums) with len(set(nums)). If they differ, duplicates exist.

Complexity Analysis

Time Complexity: *O(n)* | ***Space Complexity:*** *O(n)*

Python Solution

```
def containsDuplicate(nums):
    # Method 1: Using a set to track seen numbers
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False

def containsDuplicate_v2(nums):
    # Method 2: One-liner using set length comparison
    return len(nums) != len(set(nums))

# --- Driver Code ---
nums1 = [1, 2, 3, 1]
print(f"Input:  {nums1}")
print(f"Output: {containsDuplicate(nums1)}")

nums2 = [1, 2, 3, 4]
print(f"\nInput:  {nums2}")
print(f"Output: {containsDuplicate(nums2)}")
```

Input / Output Examples

```
Input:  [1, 2, 3, 1]
Output: True
Explanation: 1 appears at index 0 and index 3.

Input:  [1, 2, 3, 4]
Output: False
Explanation: All elements are distinct.
```

Beginner Tip: *Sets are your best friend for uniqueness checks. Lookup in a set is O(1) on average, making it perfect for duplicate detection.*

3. Best Time to Buy and Sell Stock [Easy]

Problem Statement

You are given an array **prices** where **prices[i]** is the price of a given stock on the i-th day. You want to maximize your profit by choosing a **single day to buy** and a **different day in the future to sell**. Return the maximum profit you can achieve. If no profit is possible, return 0.

Approach & Explanation

- We need to find the maximum difference between a later price and an earlier price.
- Use a **single pass** approach: track the **minimum price** seen so far.
- At each day, calculate the profit if we sold today (current price - minimum price so far).
- Keep track of the **maximum profit** encountered.
- This avoids checking every pair ($O(n^2)$) and gives us an O(n) solution.

Complexity Analysis

Time Complexity: *O(n)* | ***Space Complexity:*** *O(1)*

Python Solution

```
def maxProfit(prices):
    min_price = float('inf')  # Start with infinity
    max_profit = 0

    for price in prices:
        # Update the minimum price seen so far
        if price < min_price:
            min_price = price

        # Calculate profit if we sell today
        profit = price - min_price

        # Update max profit if current profit is better
        if profit > max_profit:
            max_profit = profit

    return max_profit

# --- Driver Code ---
prices1 = [7, 1, 5, 3, 6, 4]
print(f"Input:  {prices1}")
print(f"Output: {maxProfit(prices1)}")

prices2 = [7, 6, 4, 3, 1]
print(f"\nInput:  {prices2}")
print(f"Output: {maxProfit(prices2)}")
```

Input / Output Examples

```
Input:  [7, 1, 5, 3, 6, 4]
Output: 5
Explanation: Buy on day 2 (price=1), sell on day 5 (price=6).
```

```
            Profit = 6 - 1 = 5.

Input:  [7, 6, 4, 3, 1]
Output: 0
Explanation: Prices only decrease, so no profitable trade exists.
```

Beginner Tip: *This is a classic 'track the minimum so far' pattern. Many problems use this idea of maintaining a running minimum or maximum as you scan through an array.*

4. Valid Anagram [Easy]

Problem Statement

Given two strings **s** and **t**, return **True** if t is an **anagram** of s, and False otherwise. An anagram is a word formed by rearranging all the letters of another word, using each letter exactly once.

Approach & Explanation

- An anagram has the **exact same character frequencies** as the original word.
- Method 1: Sort both strings and compare. If they match, it is an anagram.
- Method 2 (Optimal): Use a **frequency counter** (dictionary or Counter).
- Count the frequency of each character in both strings and compare the counts.
- Python's **collections.Counter** makes this very concise.

Complexity Analysis

Time Complexity: *O(n) with Counter / O(n log n) with sorting* | ***Space Complexity:*** *O(n) for the frequency map / O(n) for sorted copies*

Python Solution

```
from collections import Counter

def isAnagram(s, t):
    # Method 1: Using Counter (frequency map)
    return Counter(s) == Counter(t)

def isAnagram_v2(s, t):
    # Method 2: Using sorting
    return sorted(s) == sorted(t)

def isAnagram_v3(s, t):
    # Method 3: Manual frequency counting
    if len(s) != len(t):
        return False

    count = {}
    for char in s:
        count[char] = count.get(char, 0) + 1
    for char in t:
        count[char] = count.get(char, 0) - 1
        if count[char] < 0:
            return False
    return True

# --- Driver Code ---
print(f"Input:  s = 'anagram', t = 'nagaram'")
print(f"Output: {isAnagram('anagram', 'nagaram')}")

print(f"\nInput:  s = 'rat', t = 'car'")
print(f"Output: {isAnagram('rat', 'car')}")
```

Input / Output Examples

```
Input:  s = 'anagram', t = 'nagaram'
Output: True
Explanation: Both have: a(3), n(1), g(1), r(1), m(1)

Input:  s = 'rat', t = 'car'
Output: False
Explanation: 'rat' has 't' but 'car' has 'c' — different letters.
```

Beginner Tip: *Counter from the collections module is extremely useful for frequency-based problems. Always consider it when comparing character or element counts.*

5. Valid Parentheses [Easy]

Problem Statement

Given a string **s** containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is **valid**. A string is valid if: (1) Open brackets must be closed by the same type of brackets. (2) Open brackets must be closed in the correct order. (3) Every close bracket has a corresponding open bracket of the same type.

Approach & Explanation

- This is a classic **stack** problem.
- Iterate through each character in the string.
- If it is an **opening bracket** ('(', '{', '['), push it onto the stack.
- If it is a **closing bracket**, check if the stack is non-empty and the top of the stack is the matching opening bracket.
- If it matches, pop the stack. If it does not match (or stack is empty), return False.
- At the end, the string is valid only if the stack is **empty** (all brackets matched).

Complexity Analysis

Time Complexity: *O(n)* | ***Space Complexity:*** *O(n)*

Python Solution

```
def isValid(s):
    # Map closing brackets to their opening counterparts
    bracket_map = {')': '(', '}': '{', ']': '['}
    stack = []

    for char in s:
        if char in bracket_map:
            # It's a closing bracket
            # Check top of stack for matching opener
            top = stack.pop() if stack else '#'
            if bracket_map[char] != top:
                return False
        else:
            # It's an opening bracket — push to stack
            stack.append(char)

    # Valid only if all brackets are matched (stack empty)
    return len(stack) == 0

# --- Driver Code ---
test1 = "()"
print(f"Input:  '{test1}'")
print(f"Output: {isValid(test1)}")

test2 = "()[]{}"
print(f"\nInput:  '{test2}'")
print(f"Output: {isValid(test2)}")
```

```
test3 = "(]"
print(f"\nInput:  '{test3}'")
print(f"Output: {isValid(test3)}")

test4 = "([{}])"
print(f"\nInput:  '{test4}'")
print(f"Output: {isValid(test4)}")
```

Input / Output Examples

```
Input:  '()'         Output: True
Input:  '()[]{}'     Output: True
Input:  '(]'         Output: False
Input:  '([{}])'     Output: True

Explanation: A stack ensures proper nesting order.
'(]' fails because ')' is expected but ']' is found.
```

Beginner Tip: *Stacks follow LIFO (Last In, First Out). They are perfect for problems involving matching, nesting, or reversal. Think 'stack' whenever you see bracket matching or undo operations.*

6. Maximum Subarray [Easy]

Problem Statement

Given an integer array **nums**, find the subarray (contiguous non-empty sequence) with the **largest sum** and return its sum. This is the classic **Kadane's Algorithm** problem.

Approach & Explanation

- This uses **Kadane's Algorithm**, a dynamic programming approach.
- Maintain two variables: **current_sum** (best sum ending at current position) and **max_sum** (global best).
- At each element, decide: either **extend** the previous subarray (add current element) or **start fresh** from current element.
- current_sum = max(num, current_sum + num) — this is the key decision.
- Update max_sum whenever current_sum exceeds it.
- The idea: if the running sum becomes negative, it is better to start a new subarray.

Complexity Analysis

Time Complexity: *O(n)* | ***Space Complexity:*** *O(1)*

Python Solution

```
def maxSubArray(nums):
    # Initialize with the first element
    current_sum = nums[0]
    max_sum = nums[0]

    # Start from index 1
    for num in nums[1:]:
        # Either extend previous subarray or start new one
        current_sum = max(num, current_sum + num)

        # Update global maximum
        max_sum = max(max_sum, current_sum)

    return max_sum

# --- Driver Code ---
nums1 = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
print(f"Input:  {nums1}")
print(f"Output: {maxSubArray(nums1)}")

nums2 = [1]
print(f"\nInput:  {nums2}")
print(f"Output: {maxSubArray(nums2)}")

nums3 = [5, 4, -1, 7, 8]
print(f"\nInput:  {nums3}")
print(f"Output: {maxSubArray(nums3)}")
```

Input / Output Examples

```
Input:  [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
Explanation: The subarray [4, -1, 2, 1] has the largest
             sum = 6.

Input:  [5, 4, -1, 7, 8]
Output: 23
Explanation: The entire array is the max subarray.
```

Beginner Tip: *Kadane's Algorithm is one of the most important algorithms to know. The core idea — 'extend or restart' — appears in many dynamic programming problems.*

7. Product of Array Except Self [Medium]

Problem Statement

Given an integer array **nums**, return an array **answer** such that **answer[i]** is equal to the product of all the elements of nums **except nums[i]**. You must write an algorithm that runs in **O(n)** time and **without using division**.

Approach & Explanation

- We cannot use division, so we need a clever approach using **prefix and suffix products**.
- **Prefix product**: For each index i, the product of all elements to the LEFT of i.
- **Suffix product**: For each index i, the product of all elements to the RIGHT of i.
- answer[i] = prefix[i] * suffix[i]
- First pass (left to right): Build prefix products in the result array.
- Second pass (right to left): Multiply each position by the running suffix product.
- This uses the output array for prefix, and a single variable for suffix — O(1) extra space.

Complexity Analysis

Time Complexity: *O(n)* | ***Space Complexity:*** *O(1) extra space (output array not counted)*

Python Solution

```
def productExceptSelf(nums):
    n = len(nums)
    answer = [1] * n

    # Pass 1: Calculate prefix products
    # answer[i] = product of all elements to the left of i
    prefix = 1
    for i in range(n):
        answer[i] = prefix
        prefix *= nums[i]

    # Pass 2: Multiply by suffix products
    # Multiply answer[i] by product of all elements to the right
    suffix = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= suffix
        suffix *= nums[i]

    return answer

# --- Driver Code ---
nums1 = [1, 2, 3, 4]
print(f"Input:  {nums1}")
print(f"Output: {productExceptSelf(nums1)}")

nums2 = [-1, 1, 0, -3, 3]
print(f"\nInput:  {nums2}")
print(f"Output: {productExceptSelf(nums2)}")
```

Input / Output Examples

```
Input:  [1, 2, 3, 4]
Output: [24, 12, 8, 6]
Explanation:
  answer[0] = 2*3*4 = 24  (all except 1)
  answer[1] = 1*3*4 = 12  (all except 2)
  answer[2] = 1*2*4 = 8   (all except 3)
  answer[3] = 1*2*3 = 6   (all except 4)

Input:  [-1, 1, 0, -3, 3]
Output: [0, 0, 9, 0, 0]
```

Beginner Tip: *The prefix/suffix product pattern is powerful. Whenever you need information about 'everything except current element,' think about building results from both directions.*

8. 3Sum [Medium]

Problem Statement

Given an integer array **nums**, return all the triplets [nums[i], nums[j], nums[k]] such that i != j, i != k, and j != k, and **nums[i] + nums[j] + nums[k] == 0**. The solution set must **not contain duplicate triplets**.

Approach & Explanation

- **Sort the array** first — this enables the two-pointer technique and easy duplicate skipping.
- Fix one number (nums[i]) and use **two pointers** (left and right) to find pairs that sum to -nums[i].
- If the three numbers sum to zero, record the triplet.
- Skip duplicate values for i, left, and right to avoid duplicate triplets.
- If the sum is too small, move left pointer right. If too large, move right pointer left.
- Time: $O(n^2)$ — sorting O(n log n) + for each element a two-pointer scan O(n).

Complexity Analysis

Time Complexity: *$O(n^2)$* | ***Space Complexity:*** *O(1) extra (ignoring output and sort space)*

Python Solution

```
def threeSum(nums):
    nums.sort()  # Sort the array first
    result = []

    for i in range(len(nums) - 2):
        # Skip duplicate values for the first number
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        # Two-pointer approach for remaining two numbers
        left, right = i + 1, len(nums) - 1

        while left < right:
            total = nums[i] + nums[left] + nums[right]

            if total == 0:
                result.append([nums[i], nums[left], nums[right]])

                # Skip duplicates for left and right
                while left < right and nums[left] == nums[left + 1]:
                    left += 1
                while left < right and nums[right] == nums[right - 1]:
                    right -= 1

                left += 1
                right -= 1
            elif total < 0:
                left += 1   # Need a larger sum
            else:
                right -= 1  # Need a smaller sum
```

```
    return result

# --- Driver Code ---
nums1 = [-1, 0, 1, 2, -1, -4]
print(f"Input:  {nums1}")
print(f"Output: {threeSum(nums1)}")

nums2 = [0, 1, 1]
print(f"\nInput:  {nums2}")
print(f"Output: {threeSum(nums2)}")
```

Input / Output Examples

```
Input:  [-1, 0, 1, 2, -1, -4]
Output: [[-1, -1, 2], [-1, 0, 1]]
Explanation:
  -1 + (-1) + 2 = 0
  -1 + 0 + 1 = 0

Input:  [0, 1, 1]
Output: []
Explanation: No triplet sums to 0.
```

Beginner Tip: *The 'sort + two pointers' pattern is essential for k-Sum problems. Sorting lets you control the direction of your search and easily skip duplicates.*

9. Merge Intervals [Medium]

Problem Statement

Given an array of **intervals** where intervals[i] = [start_i, end_i], merge all **overlapping intervals**, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Approach & Explanation

- **Sort** the intervals by their start time.
- Initialize the merged list with the first interval.
- For each subsequent interval, check if it **overlaps** with the last merged interval.
- Overlap condition: current start <= last merged end.
- If overlapping, **merge** by extending the end to max(last_end, current_end).
- If not overlapping, add the current interval as a new entry.

Complexity Analysis

Time Complexity: *O(n log n) due to sorting* | ***Space Complexity:*** *O(n) for the merged output*

Python Solution

```
def merge(intervals):
    # Sort by start time
    intervals.sort(key=lambda x: x[0])

    merged = [intervals[0]]

    for current in intervals[1:]:
        last = merged[-1]

        if current[0] <= last[1]:
            # Overlapping — merge by extending the end
            merged[-1][1] = max(last[1], current[1])
        else:
            # Non-overlapping — add as new interval
            merged.append(current)

    return merged

# --- Driver Code ---
intervals1 = [[1,3], [2,6], [8,10], [15,18]]
print(f"Input:  {intervals1}")
print(f"Output: {merge(intervals1)}")

intervals2 = [[1,4], [4,5]]
print(f"\nInput:  {intervals2}")
print(f"Output: {merge(intervals2)}")
```

Input / Output Examples

```
Input:  [[1,3], [2,6], [8,10], [15,18]]
Output: [[1,6], [8,10], [15,18]]
```

```
Explanation: [1,3] and [2,6] overlap -> merged to [1,6].

Input:  [[1,4], [4,5]]
Output: [[1,5]]
Explanation: [1,4] and [4,5] touch at 4 -> merged to [1,5].
```

Beginner Tip: *Sorting is almost always the first step in interval problems. Once sorted, you only need to compare each interval with the previous one — the problem becomes linear.*

10. Group Anagrams [Medium]

Problem Statement

Given an array of strings **strs**, group the **anagrams** together. You can return the answer in any order. An anagram is a word or phrase formed by rearranging the letters, using all the original letters exactly once.

Approach & Explanation

- Two strings are anagrams if they have the **same sorted characters**.
- Use a **dictionary** where the key is the sorted version of each string.
- All anagrams will map to the **same key**.
- Iterate through all strings, sort each one, and group them by their sorted key.
- Python's **defaultdict(list)** simplifies the grouping.
- Alternative key: use a tuple of character counts (avoids sorting each string).

Complexity Analysis

Time Complexity: *O(n * k log k) where n = number of strings, k = max string length* | ***Space Complexity:*** *O(n * k) for storing all strings in the map*

Python Solution

```
from collections import defaultdict

def groupAnagrams(strs):
    # Dictionary: sorted string -> list of anagrams
    anagram_map = defaultdict(list)

    for s in strs:
        # Sort the string to create the key
        key = ''.join(sorted(s))
        anagram_map[key].append(s)

    return list(anagram_map.values())

def groupAnagrams_v2(strs):
    # Alternative: Use character count tuple as key
    anagram_map = defaultdict(list)

    for s in strs:
        # Count frequency of each letter (a-z)
        count = [0] * 26
        for char in s:
            count[ord(char) - ord('a')] += 1
        key = tuple(count)
        anagram_map[key].append(s)

    return list(anagram_map.values())

# --- Driver Code ---
strs1 = ["eat","tea","tan","ate","nat","bat"]
print(f"Input:  {strs1}")
```

```
print(f"Output: {groupAnagrams(strs1)}")

strs2 = [""]
print(f"\nInput:  {strs2}")
print(f"Output: {groupAnagrams(strs2)}")
```

Input / Output Examples

```
Input:  ['eat','tea','tan','ate','nat','bat']
Output: [['eat','tea','ate'], ['tan','nat'], ['bat']]
Explanation:
  'eat', 'tea', 'ate' are anagrams (sorted: 'aet')
  'tan', 'nat' are anagrams (sorted: 'ant')
  'bat' has no anagram partner

Input:  ['']
Output: [['']]
```

Beginner Tip: *When you need to group items by some property, use a dictionary with the property as the key. Sorting a string creates a 'canonical form' that all anagrams share.*

11. Maximum Product Subarray (Optional) [Medium]

Problem Statement

Given an integer array **nums**, find a contiguous non-empty subarray that has the **largest product**, and return the product. The array may contain positive numbers, negative numbers, and zeros.

Approach & Explanation

- Similar to Maximum Subarray, but multiplication has a twist: **two negatives make a positive**.
- Track both the **current maximum** and **current minimum** product ending at each position.
- Why track minimum? A large negative number multiplied by another negative can become the largest positive.
- At each step: new_max = max(num, max_so_far * num, min_so_far * num)
- Similarly: new_min = min(num, max_so_far * num, min_so_far * num)
- Update the global maximum product after each step.

Complexity Analysis

Time Complexity: *O(n)* | ***Space Complexity:*** *O(1)*

Python Solution

```
def maxProduct(nums):
    # Track both max and min products
    current_max = nums[0]
    current_min = nums[0]
    result = nums[0]

    for num in nums[1:]:
        # Store current_max before overwriting
        temp_max = current_max

        # Max could come from: num itself, extending max, or
        # extending min (if both num and min are negative)
        current_max = max(num, temp_max * num, current_min * num)
        current_min = min(num, temp_max * num, current_min * num)

        result = max(result, current_max)

    return result

# --- Driver Code ---
nums1 = [2, 3, -2, 4]
print(f"Input:  {nums1}")
print(f"Output: {maxProduct(nums1)}")

nums2 = [-2, 0, -1]
print(f"\nInput:  {nums2}")
print(f"Output: {maxProduct(nums2)}")

nums3 = [-2, 3, -4]
print(f"\nInput:  {nums3}")
print(f"Output: {maxProduct(nums3)}")
```

Input / Output Examples

```
Input:  [2, 3, -2, 4]
Output: 6
Explanation: Subarray [2, 3] has the largest product = 6.

Input:  [-2, 0, -1]
Output: 0
Explanation: Best we can do is 0 (the element itself).

Input:  [-2, 3, -4]
Output: 24
Explanation: (-2) * 3 * (-4) = 24 — all elements!
```

Beginner Tip: *When dealing with products and negative numbers, always track both the minimum and maximum. This dual-tracking pattern is essential for product-based DP problems.*

12. Search in Rotated Sorted Array (Optional) [Medium]

Problem Statement

An integer array **nums** sorted in ascending order is **rotated** at some pivot. For example, [0,1,2,4,5,6,7] might become [4,5,6,7,0,1,2]. Given the rotated array and a **target** value, return its index if found, or **-1** if not. Your algorithm must run in **O(log n)** time.

Approach & Explanation

- Use a modified **Binary Search**.
- At each step, find the mid point. One half of the array is always sorted.
- Determine **which half is sorted** by comparing nums[left] with nums[mid].
- If the **left half is sorted** (nums[left] <= nums[mid]):
- Check if target falls in range [nums[left], nums[mid]]. If yes, search left; else search right.
- If the **right half is sorted**:
- Check if target falls in range [nums[mid], nums[right]]. If yes, search right; else search left.
- Continue until found or search space is exhausted.

Complexity Analysis

Time Complexity: *O(log n)* | ***Space Complexity:*** *O(1)*

Python Solution

```
def search(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2

        # Found the target
        if nums[mid] == target:
            return mid

        # Determine which half is sorted
        if nums[left] <= nums[mid]:
            # Left half is sorted
            if nums[left] <= target < nums[mid]:
                right = mid - 1  # Target is in sorted left half
            else:
                left = mid + 1   # Target is in right half
        else:
            # Right half is sorted
            if nums[mid] < target <= nums[right]:
                left = mid + 1   # Target is in sorted right half
            else:
                right = mid - 1  # Target is in left half

    return -1  # Target not found
```

```
# --- Driver Code ---
nums1 = [4, 5, 6, 7, 0, 1, 2]
target1 = 0
print(f"Input:  nums = {nums1}, target = {target1}")
print(f"Output: {search(nums1, target1)}")

nums2 = [4, 5, 6, 7, 0, 1, 2]
target2 = 3
print(f"\nInput:  nums = {nums2}, target = {target2}")
print(f"Output: {search(nums2, target2)}")

nums3 = [1]
target3 = 0
print(f"\nInput:  nums = {nums3}, target = {target3}")
print(f"Output: {search(nums3, target3)}")
```

Input / Output Examples

```
Input:  nums = [4,5,6,7,0,1,2], target = 0
Output: 4
Explanation: 0 is at index 4.

Input:  nums = [4,5,6,7,0,1,2], target = 3
Output: -1
Explanation: 3 is not in the array.

Input:  nums = [1], target = 0
Output: -1
```

Beginner Tip: *Modified binary search is a must-know pattern. The key insight is that in a rotated sorted array, at least one half is always sorted. Use that sorted half to decide your search direction.*

Quick Reference Cheat Sheet

Problem	Pattern	Time	Space	Key Insight
Two Sum	Hash Map	O(n)	O(n)	Complement lookup
Contains Duplicate	Set	O(n)	O(n)	Set membership
Buy/Sell Stock	Running Min	O(n)	O(1)	Track min price
Valid Anagram	Freq Count	O(n)	O(n)	Counter equality
Valid Parentheses	Stack	O(n)	O(n)	LIFO matching
Max Subarray	Kadane's	O(n)	O(1)	Extend or restart
Product Except Self	Prefix/Suffix	O(n)	O(1)*	Two-pass multiply
3Sum	Sort+2 Ptr	$O(n^2)$	O(1)*	Fix one, find two
Merge Intervals	Sort+Merge	O(n log n)	O(n)	Compare ends
Group Anagrams	Hash Map	O(nk log k)	O(nk)	Sorted key
Max Product Sub.	Dual Track	O(n)	O(1)	Track min & max
Search Rotated	Binary Search	O(log n)	O(1)	Sorted half check

** O(1) extra space — output array not counted.*

Study Tips for Beginners

- **Understand before memorizing.** Focus on WHY each approach works, not just the code.
- **Identify the pattern.** Each problem teaches a reusable pattern — learn to recognize them.
- **Start with brute force.** Always think of the naive solution first, then optimize.
- **Dry run with examples.** Trace through the code with small inputs on paper.
- **Practice variations.** Once you solve a problem, try it with different constraints.
- **Time yourself.** Aim to solve Easy problems in 15 min and Medium in 25-30 min.